Open-Ended Questions

1. What is the purpose of procedures in your application?
   - Procedures divide the assembly program into logical, reusable subroutines such as display_welcome, validate_birthdate, battle_enemy, and print_string so each major task is isolated and self-contained. Instead of repeating system call boilerplate (mov eax, 4; mov ebx, 1; int 0x80) dozens of times throughout _start, calling print_string or read_line centralizes input/output operations. This structure improves code readability, simplifies debugging, and allows menus and loops to invoke complex operations with a single call instruction.

2. How do CALL and RET control program execution?
   - When the CPU executes a CALL instruction (such as call validate_birthdate), it pushes the address of the next instruction (EIP) onto the stack and loads the target procedure's address into EIP, transferring execution to the procedure. Inside the procedure, push ebp and mov ebp, esp establish a stable stack frame. When the procedure finishes and executes RET, the CPU pops the saved return address from the top of the stack back into EIP, seamlessly resuming execution at the instruction immediately following the original CALL.

3. Which conditional jump instructions did you use, and why?
   - Both applications use JE (Jump if Equal) and JNE (Jump if Not Equal) after CMP instructions to match exact menu selections, character classes, and Y/N prompts (for example, cmp eax, 1; je .opt_love and cmp dword [date_valid], 1; jne .date_input_loop). For range validation, date cutoffs, and combat health checks, the programs use signed comparison jumps: JL (Jump if Less), JG (Jump if Greater), JLE (Jump if Less or Equal), and JGE (Jump if Greater or Equal). For instance, JL and JG detect out-of-range months (< 1 or > 12), while JLE checks zodiac cutoff days (cmp ebx, 22; jle .set_leo) and detects when health or potion counts drop to 0 or below (cmp dword [hero_hp], 0; jle .hero_defeated).

4. How does your program handle invalid user input?
   - All numeric user input is first processed by the parse_int helper procedure, which inspects each ASCII byte to verify it falls between '0' and '9' (cmp cl, '0'; jl .invalid_num and cmp cl, '9'; jg .invalid_num) and returns -1 in EAX if the input is empty or contains non-digit characters. Validation procedures and menu handlers then test the parsed integer against valid boundaries using CMP and conditional jumps. When an invalid month, day, class choice, or menu option is detected, the program prints a clear error message (err_month, err_day, err_class, or err_menu) and jumps back to the prompt or menu loop so the user can try again without crashing the program.

5. What would be harder to maintain without procedures?

   - Without procedures, the entire program would be a single monolithic block of instructions filled with hundreds of overlapping labels and duplicated int 0x80 system call setups. Re-displaying zodiac information from the menu in Machine Problem 1 or calling use_potion from both the Main Menu and the Combat Menu in Machine Problem 2 would require duplicating identical blocks of assembly code in multiple places. Furthermore, tracking register states (EAX, EBX, ECX, EDX) across input parsing, integer printing, and branching without isolated procedure stack frames would dramatically increase the risk of register corruption and infinite loops.

Application-Specific Open-Ended Questions

Answer all questions for the application you developed. Write 3–5 complete sentences per question, refer to specific procedures or instructions from your own source code, and provide examples where requested.

Machine Problem 1 — Zodiac Horoscope and Fortune Teller

1. Explain how your determine_zodiac procedure identifies the correct sign from the birth month and day. Describe at least two conditional comparisons.

- The determine_zodiac procedure uses the birth month and day to identify the zodiac sign. It gets the correct cutoff date from cutoff_tbl and compares it with the birth day using CMP edx, eax and JLE .before_cutoff. If the day is before or equal to the cutoff, it uses before_tbl; otherwise, it uses after_tbl. The selected zodiac index is then stored in the sign variable.

2. How does your program handle zodiac date ranges that cross two months, such as Capricorn? Explain the branching logic.

- The program uses different cutoff values for every month to handle zodiac signs that cross two months. For example, in January, days 1 to 19 are assigned to Capricorn, while days after 19 are assigned to Aquarius. In December, days 1 to 21 are Sagittarius, while days after 21 are Capricorn. The before_tbl and after_tbl tables control these branches.

3. Why did you divide input collection, date validation, zodiac identification, and horoscope display into separate procedures?

- I divided the program into separate procedures to make the code organized and easier to understand. The get_user_info procedure collects the user's details, while validate_birthdate checks if the date is valid. The determine_zodiac procedure identifies the sign, and display_zodiac_info displays the result. This makes the program easier to debug and maintain.

4. Describe how your program rejects invalid dates such as February 30 and an invalid month such as 13.

- The validate_birthdate procedure first checks if the month is between 1 and 12. It uses JL and JG to reject values such as 13. It then uses days_tbl to check the maximum number of days in the selected month, so February 30 is rejected. When the input is invalid, an error message is shown and the program asks the user to enter the date again.

5. How does the program select a horoscope category after the user enters a menu choice? Identify the conditional jump instructions used.

- The program reads the menu choice using the read_int procedure and stores the value in EAX. It uses CMP and JE to check choices 1 to 6. Choice 1 displays the love horoscope, choice 2 displays the career horoscope, and choice 3 displays the health horoscope. Choices 4, 5, and 6 call show_lucky, display_zodiac_info, and exit_program.

6. How did you implement the zodiac ASCII art? Explain how the correct design is selected and displayed.

- I created separate ASCII art designs for each zodiac sign and stored them in the data section. Their addresses are placed in the art_tbl table. The sign variable contains the current zodiac index, which is used by display_zodiac_art to select the correct design. The selected art is displayed using the print_z procedure.

7. What happens when a user enters an invalid menu option? Explain how control returns to the menu.

- When the user enters an invalid menu option, the program checks the value using CMP, JL, and JG. If the value is not from 1 to 6, control goes to the bad_choice label. The program displays the err_choice message and uses JMP menu_loop to return to the menu. This allows the user to enter another choice without restarting the program.

8. If you added a new horoscope category, which procedures and conditional branches would you modify? Explain why.

- If I added a new horoscope category, I would first add the new option to menu_text. I would then add another CMP and JE branch in the main menu loop to recognize the new choice. I would also add a new horoscope message and modify show_horoscope or create a separate procedure for the new category. The existing input validation and zodiac identification procedures would not need to change.

Machine Problem 2 — Dungeon Adventure and Survival Game

1. Explain how initialize_character assigns different health, attack, and potion values for Warrior, Mage, and Archer.

In initialize_character, the procedure first resets [hero_gold] to 0 and [current_room] to 1, then loads [hero_class] into EAX. It uses cmp eax, 1; je .init_warrior and cmp eax, 2; je .init_mage (with .init_archer as the third branch) to jump to the appropriate stat-assignment block. Inside .init_warrior, mov dword instructions set [hero_hp] and [hero_max_hp] to 100, [hero_atk] to 20, and [hero_potions] to 2; .init_mage assigns 70 HP, 30 Attack, and 3 Potions; and .init_archer assigns 85 HP, 25 Attack, and 2 Potions.

2. Describe the procedures and conditional jumps used when the player chooses Attack, Defend, Use Potion, or Run.

Inside battle_enemy, the player's combat input is parsed into EAX and compared against 1 through 4 using CMP and JE (je .act_attack, je .act_defend, je .act_potion, and je .act_run). Choosing Attack (.act_attack) subtracts [hero_atk] from [enemy_hp] and uses cmp dword [enemy_hp], 0; jle .enemy_defeated before jumping to .enemy_turn. Choosing Defend (.act_defend) sets [defend_bonus] to 5 and jumps to .enemy_turn, whereas choosing Use Potion (.act_potion) calls the use_potion procedure and choosing Run (.act_run) prints msg_run_away and jumps to .battle_end to return to the main menu.

3. How does your battle_enemy procedure determine whether the player or enemy has been defeated? Explain the comparisons.

After the player attacks in .act_attack, the instruction sub [enemy_hp], eax reduces the enemy's health, followed immediately by cmp dword [enemy_hp], 0 and jle .enemy_defeated. Using JLE (Jump if Less Than or Equal) ensures that if the enemy's health drops to 0 or becomes negative, it is clamped to 0 (mov dword [enemy_hp], 0) and the enemy cannot retaliate. Similarly, after the enemy retaliates in

.apply_enemy_dmg, sub [hero_hp], eax is followed by cmp dword [hero_hp], 0 and jle .hero_defeated, which clamps [hero_hp] to 0, sets [game_status] to 2, and calls display_game_result.

4. Explain how you prevent health from exceeding the maximum and prevent potion use when the inventory is empty.

At the beginning of use_potion, the program executes cmp dword [hero_potions], 0 followed by jle .no_potions; if no potions remain, it prints "Error: No health potions remaining!" and skips both the potion decrement and health addition. When a potion is available, the procedure executes dec dword [hero_potions] and add dword [hero_hp], 20. It then loads [hero_hp] into EAX and compares it with [hero_max_hp] using cmp eax, [hero_max_hp]; jle .hp_capped; if [hero_hp] exceeds [hero_max_hp], mov eax, [hero_max_hp] and mov [hero_hp], eax cap the health at the character's starting maximum.

5. How is the current dungeon room tracked, and what conditions allow the player to advance to the next room?

The current room is stored in the 32-bit .bss variable [current_room], which initialize_character initializes to 1. In explore_dungeon, .room_dispatch compares [current_room] against 1, 2, and 3 using CMP and JE. In Room 1, [current_room] only increments (inc dword [current_room]) inside .enemy_defeated when [enemy_hp] reaches 0 or less; if the player selects Run ([4]), battle_enemy exits without incrementing [current_room], keeping the player in Room 1. In Room 2, selecting either [1] Open Chest or [2] Leave Chest sets mov dword [current_room], 3 and advances to the Dragon's Lair.

6. What is the purpose of separating display_status, display_inventory, and battle_enemy into different procedures?

Separating display_status, display_inventory, and battle_enemy keeps UI presentation logic isolated from combat state mutations. A player can inspect their health and attack via display_status (Main Menu option [2]) or check their gold and potions via display_inventory (Main Menu option [4]) between rooms without triggering combat. This separation also makes debugging easier because each procedure manages its own stack frame and output formatting independently.

7. How does your program preserve important register values across procedures and Linux system calls? Give one example.

Every procedure preserves the caller's stack frame using push ebp; mov ebp, esp on entry and mov esp, ebp; pop ebp before ret, while utility procedures push and pop registers modified by int 0x80. For example, in print_string, the procedure executes push eax, push ebx, push ecx, and push edx before invoking sys_write (mov eax, 4; mov ebx, 1; int 0x80), and restores them in reverse order with pop before returning. Additionally, in .apply_enemy_dmg inside battle_enemy, the calculated enemy damage in EAX is saved on the stack using push eax across call print_string and call print_int, then restored with pop eax so sub [hero_hp], eax subtracts the exact damage amount.

8. If you introduced a fourth room with a stronger enemy, what program data, procedures, and conditions would need to change?

In the .data section, I would add a new ASCII art string and room banner for Room 4 (for example, lich_art). In explore_dungeon, I would add cmp eax, 4; je .room_4 under .room_dispatch, and inside

.room_4 I would print the new ASCII art, set higher [enemy_hp] and [enemy_atk] values (e.g., 100 HP and 20 ATK), and call battle_enemy. Finally, inside .enemy_defeated in battle_enemy, I would change the victory check from cmp dword [current_room], 3 to cmp dword [current_room], 4 so defeating the Dragon in Room 3 increments [current_room] to 4 instead of ending the game prematurely.